

```
27     }
28
29     copy_page_to_user_buffer(buf->page, buf->offset, chars, to);
30     ret += chars;
31     buf->offset += chars;
32     buf->len -= chars;
33
34     if (!buf->len) {
35         release_pipe_buf(pipe, buf);
36         tail++;
37         pipe->tail = tail;
38     }
39     total_len -= chars;
40     if (!total_len)
41         break;
42 }
43 // 没有数据，阻塞等待（或直接返回错误信息）
44 ...
45 }
46 pipe_unlock(pipe); // 释放管道锁
47 // ...
48 }
```

为了从缓冲区中读取数据，首先内核会锁住整个管道，避免在读的过程中发生管道状态的变化。随后，内核尝试从缓冲区中读取数据。如果当前有数据，那么内核会在读取到足够的数据后返回。如果当前没有数据，那么内核会尝试唤醒等待的写者，让其开始写入数据，并且使自己陷入阻塞状态。在进入阻塞状态前需要释放管道锁，否则写者即使被唤醒也无法进行相应的操作。当写者完成写操作后，会唤醒当前等待的读者，使其开始尝试新一轮的读操作。管道写和管道读的过程十分类似。

8.2.3 命名管道和匿名管道

管道的连接是如何建立的呢？在经典的 UNIX 实现中，管道通常有两类——命名管道和匿名管道，主要区别在于它们的创建方式。匿名管道是通过 `pipe` 系统调用创建的，在创建的同时进程会拿到读写的端口（两个文件描述符）。由于整个管道没有全局的“名字”，因此只能通过这两个文件描述符来使用它。这种情况下，通常要结合 `fork` 来使用，即用继承的方式来建立父子进程间的连接。具体过程是，父进程首先通过 `pipe` 创建对应的管道的两端，然后通过 `fork` 创建子进程。由于子进程可以继承文件描述符，父子进程相当于通过 `fork` 的继承完成了一次 IPC 权限的分发。后续父子进程就可以通过管道来进行进程间的通信。要注意的是，在完成继承后，其实父子进程会同时拥有管道的两端，此时需要父子进程主动关闭多余的端口，否则可能导致通信出错。这种连接方式对于父子进程等有着创建关系的进程来说比较方便，但是对于两个关系较远的进程就不太适用。

命名管道可以解决这个问题。命名管道是由另一个命令 `mkfifo` 来创建的。创建过程中会指定一个全局的文件名，由这个文件名（如 `/tmp/namedpipe`）来指代一个具体的管道（即管道名）。通过这种方式，只要两个进程通过一个相同的管道名进行创建（并且都拥有对其的访问权限），就可以实现在任意两个进程间建立管道的通信连接。

练习

8.6 管道的使用中是否包含前面介绍的命名服务？请解释理由。

8.7 管道属于间接通信还是直接通信？请解释理由。

8.3 内存接口 IPC：共享内存

本节主要知识点

□ 基于共享内存的通信机制需要内核提供怎样的支持？

□ 基于共享内存的单生产者单消费者是如何实现的？

为什么需要共享内存？共享内存存在本章开头的例子中已经使用到了，本节将进一步介绍共享内存的内部结构设计和接口。使用共享内存的很重要的一个原因是性能。其他进程间通信机制，包括消息队列（下节介绍）、管道等，都依赖内核提供完整的缓冲数据、接收消息、发送消息等一系列进程间通信接口。虽然这些完善的抽象方便了用户进程的使用，但其中涉及的数据拷贝和控制流转移等处理逻辑影响了这些抽象的性能。共享内存的思路其实是内核为需要通信的进程建立共享区域。一旦共享区域建立完成，内核基本上不需要参与进程间通信。通信的多方间既可以直接使用共享区域上的数据，也可以将共享区域当成消息缓冲区。大部分微内核系统都支持共享内存机制。本节将介绍 System V 共享内存。

8.3.1 共享内存

共享内存的核心思路其实很简单：共享内存允许两个或多个进程在其所在虚拟地址空间中映射相同的物理内存页，从而进行通信。本节以 Linux 中的设计为例（如图 8.7 所示）来介绍一些细节的设计和实现。

首先，内核会为全局所有的共享内存维护一个全局的队列结构，也即图中的共享内存队列。这个队列的每一项（`shmid_kernel` 结构体）都是和一个 IPC key 绑定的，这和其他 System V 的 IPC 机制类似。进程可以通过同样的 key 来找到并使用同一段共享内存区域。虽然这样的 key 是全局唯一的，但是能否使用这段共享内存，是通过 System V

的权限检查机制来判断的，这在前文中已经有所介绍。只要进程有对应的权限，就能够通过内核接口（`shmat`）将一段共享内存的区域映射到自己的虚拟地址空间中。每段共享内存是由结构体 `shmid_kernel` 封装的，其包含一个 `file` 类型的结构体。这是因为在 Linux 的系统设计中，将共享内存的机制封装在了一个特殊的文件系统上^①。这个 `file` 类型的结构体通过文件系统最终指向一段共享内存页的集合，这些共享内存页就是这段共享内存对应的物理内存。存在这么多层的封装，除了便于利用 Linux 内核里的现有其他组件的功能（如文件系统）外，也是为了支持共享内存的换页（Swap）和内存页动态分配（Demand Paging）。

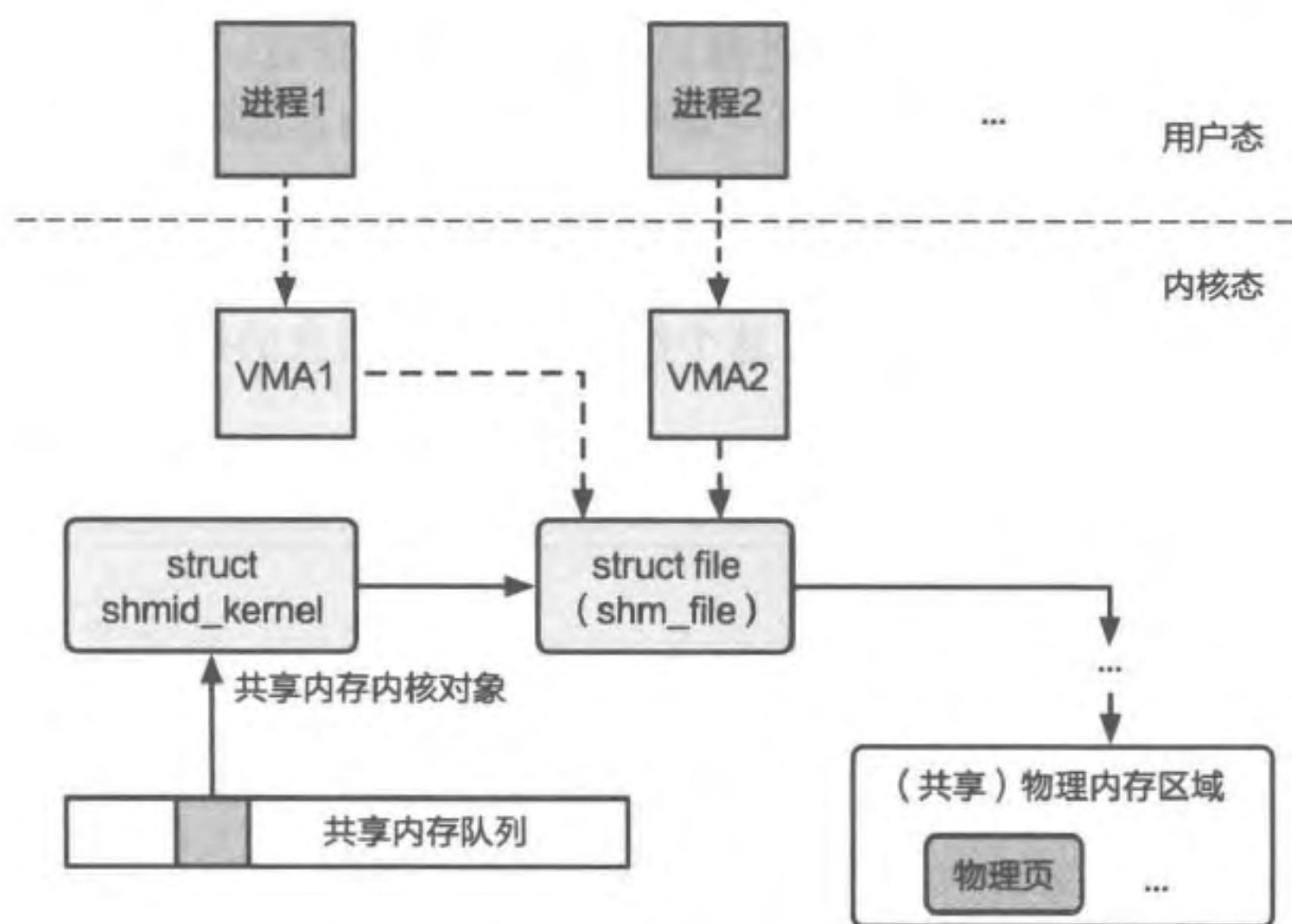


图 8.7 共享内存

如图 8.7 所示，进程 1 和进程 2 分别对同一个共享内存建立了映射（`shmat`）之后，内核会为它们分配两个 VMA（Virtual Memory Area）结构体，让它们都指向 `file`。这里的 VMA 会描述进程的一段虚拟地址空间的映射。建立了这两个 VMA，内核就能够从用户进程的虚拟地址找到对应的 VMA，从而知道这是一个共享内存的区间。值得注意的是，两个 VMA 对应的虚拟地址可以是不同的，这并不影响它们映射到相同的物理内存。另外，虽然图中只有两个进程，但是该机制天然地支持任意数量的进程来共享同一个共享内存区域，只要为它们分配指向 `file` 结构的 VMA 即可。

当进程不再希望共享虚拟内存时，可以将共享的内存从虚拟内存中取消映射（`shmdt` 接口）。这里取消映射的操作（`detach` 操作）只影响当前进程的映射，其他仍在使用共

① 即共享内存文件系统（shm 文件系统），在实际中使用 `tmpfs` 实现。

享内存的进程是不受影响的。

8.3.2 基于共享内存的进程间通信

在简单 IPC 方案中，我们已经看到共享内存是如何帮助两个进程进行通信的。基于共享内存的进程间通信的一个特点是操作系统在通信过程中不会干预数据传输。从操作系统的角度来看，共享内存为两个（或多个）进程在它们的（虚拟）地址空间中映射了同一段物理内存。进程基于这段共享内存设计通信方案，操作系统通常不参与后续的通信过程。

这一小节将介绍基于共享内存的进程间通信的一个经典问题——生产者 - 消费者问题。即存在两个进程，一个是生产者进程，负责产生新的信息，一个是消费者进程，负责消耗信息。生产者和消费者需要共享一块缓冲区（共享内存的用户态抽象）。

代码片段 8.11 给出了该问题中基于共享内存通信的数据结构。其中，buffer 对应的是共享缓冲区。这里我们给定了缓冲区的大小，即 BUFFER_SIZE 个元素。缓冲区中的每个元素由一个结构体 item 表示，这个结构体对应着消息的抽象。

代码片段 8.11 共享内存通信的实现：共享数据结构。这个数据结构和简单 IPC 中的设计类似，但是扩展了缓冲区的大小，并且发送者和接收者也不会固定在特定的缓冲区序号上

```

1 #define BUFFER_SIZE 10
2 typedef struct {
3     struct msg_header header;
4     char data[0];
5 } item;
6
7 item buffer[BUFFER_SIZE];
8 volatile int buffer_write_cnt = 0;
9 volatile int buffer_read_cnt = 0;
10 volatile int empty_slot = BUFFER_SIZE;
11 volatile int filled_slot = 0;

```

有了共享的缓冲区 buffer 之后，生产者的任务就是在缓冲区上产生新的数据。在代码片段 8.12 中，empty_slot 和 filled_slot 分别表示当前在缓冲区上空置的和包含消息的区域的个数。而 buffer_write_cnt 则表示生产者放置新消息的位置。生产者会首先判断当前是否存在空闲的缓存区域，如果有，则写入一个消息，并且对应地修改 empty_slot、filled_slot 和 buffer_write_cnt。注意，这里只考虑单个生产者和单个消费者的情况，对于多线程等情况将在后续章节（同步原语）中展开介绍。

代码片段 8.12 共享内存通信的实现：生产者

```

1 // 生成新的消息
2 int send(item msg)
3 {
4     while (empty_slot == 0)

```

```

5      ; // 没有空闲缓冲区
6      empty_slot--;
7      buffer[buffer_write_cnt] = msg; // msg 的类型是 item
8      buffer_write_cnt = (buffer_write_cnt + 1) % BUFFER_SIZE;
9      filled_slot++;
10     ...
11 }

```

消费者的任务是从缓冲区中获取数据。类似地，在代码片段 8.13 中，通过检查 `filled_slot` 的值，消费者可以判断是否有未处理的消息。对于消费者而言，使用另一个变量 `buffer_read_cnt` 来表示下一个可以读取消息的位置。`recv` 函数会从缓冲区中取出下一个未处理的消息，并对应地修改 `filled_slot`、`empty_slot` 和 `buffer_read_cnt` 的值。

代码片段 8.13 共享内存通信的实现：消费者

```

1 item recv(void)
2 {
3     ... //msg 的类型是 item
4     while (filled_slot == 0)
5         ; // 没有未处理消息
6
7     filled_slot--;
8
9     // 将一个消息从缓冲区中移出
10    msg = buffer[buffer_read_cnt];
11    buffer_read_cnt = (buffer_read_cnt + 1) % BUFFER_SIZE;
12    empty_slot++;
13    return msg;
14 }

```

8.4 消息接口 IPC：消息队列

本节主要知识点

- 消息队列中的消息抽象是怎么设计的？
- 为什么消息队列需要支持类型？

为什么需要消息队列？ 相比本章介绍的其他通信机制，消息队列是唯一以消息为（内核提供的）数据抽象的通信方式。应用可以通过消息队列来发送消息以及接收消息。发送和接收的接口是内核提供的。此外，消息队列是一种非常灵活的通信机制，支持多个发送者和接收者同时存在。Linux 还为消息队列中的每个消息提供了类型的抽象，使得消息的发送者和接收者可以根据类型来选择性地处理消息。本节将介绍 System V 消息队列。